Documentation Addendum — Phase 1 Improvements

March 2026 | Supplements: DEVELOPER_GUIDE, DATABASE_GUIDE, OPERATIONS_GUIDE, TESTING_AND_DEPLOYMENT_GUIDE

> This document records everything added or changed during the Phase 1 improvement sprint that is not yet reflected in the existing guides. When those guides are next revised, this addendum should be merged in and then deleted.

---

1. New Environment Variables

1.1 Frontend (`frontend/.env.local` / Vercel Dashboard)

The following variables are **new** and not listed in `DEVELOPER_GUIDE.md §Environment Variables Reference`.

Variable	Required	Description
`SANITY_WEBHOOK_SECRET`	Yes (prod)	HMAC-SHA256 secret for verifying incoming Sanity webhook payloads. Set the same value in manage.sanity.io → your project → API → Webhooks.
`LOOPS_API_KEY`	Yes (prod)	API key from app.loops.so → Settings → API. Server-side only — never expose to the browser.
`LOOPS_TEMPLATE_WELCOME`	No	Loops transactional template ID for welcome emails. Leave blank to fall back to legacy HTML send.
`LOOPS_TEMPLATE_DIGEST`	No	Template ID for weekly digest emails.
`LOOPS_TEMPLATE_PAYMENT_FAILED`	No	Template ID for failed payment notifications.
`LOOPS_TEMPLATE_TRIAL_ENDING`	No	Template ID for trial-ending reminders.
`LOOPS_TEMPLATE_SURVEY_RESULTS`	No	Template ID for survey result delivery emails.
`NEXT_PUBLIC_SENTRY_DSN`	Yes (prod)	Sentry DSN for frontend error capture. Found in sentry.io → your project → Settings → Client Keys.
`SENTRY_ORG`	Yes (prod)	Your Sentry organization slug (used for source map uploads during build).
`SENTRY_PROJECT`	Yes (prod)	Your Sentry project slug. Convention: `htr-platform`.
`SENTRY_AUTH_TOKEN`	Yes (prod)	Sentry auth token for source map uploads. Generate at sentry.io → Settings → Auth Tokens.

1.2 Backend (`backend/.env` / Railway Dashboard)

No new backend variables were added. The existing `INGEST_SECRET` variable now also protects the new `GET /api/ingest/status` endpoint (same Bearer token — see §3.3 below).

---

2. New Database Tables (Supabase Migrations)

The following tables were added via migrations and are **not yet documented in `DATABASE_GUIDE.md`**.

2.1 `rag_query_log` (Migration 015)

Logs every AI chat query for evaluation, content gap analysis, and latency monitoring.

```sql
CREATE TABLE rag_query_log (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id             UUID REFERENCES auth.users(id) ON DELETE SET NULL,
    session_id          TEXT,
    query               TEXT NOT NULL,
    role                TEXT,                  -- user's tier at query time
    model_used          TEXT,
    retrieved_doc_ids   TEXT[],               -- IDs of retrieved rag_documents
    retrieved_scores    FLOAT[],              -- cosine similarity scores
    response_preview    TEXT,                 -- first 500 chars of response
    latency_ms          INT,
    tokens_used         INT,
    was_zero_result     BOOLEAN NOT NULL DEFAULT FALSE,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

RLS: Users can read only their own rows. Service role (backend) can read all.

Admin view: A pre-built aggregate view `rag_query_stats` is available:

```sql
SELECT * FROM rag_query_stats;
-- Returns: day, total_queries, zero_result_queries, avg_latency_ms,
p95_latency_ms, unique_users
```

This is the primary operational dashboard for RAG quality monitoring. Query it weekly to identify content gaps (high `zero_result_queries`) and latency regressions (`p95_latency_ms`).

2.2 `bookmarks` (Migration 016)

Stores per-user article bookmarks. Tied to Sanity document identity (`sanity_id` + `slug`).

```sql
CREATE TABLE bookmarks (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id      UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
    sanity_id    TEXT NOT NULL,          -- Sanity document _id
    slug         TEXT NOT NULL,          -- URL slug
    title        TEXT NOT NULL,
    pillar       TEXT,                   -- 'policy', 'economics', etc.
    content_type TEXT,                   -- 'policyAnalysis', 'caseStudy', etc.
    note         TEXT CHECK (char_length(note) <= 2000),
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (user_id, sanity_id)
);
```

RLS: Full access (SELECT/INSERT/UPDATE/DELETE) scoped to `user_id = auth.uid()`.

UI entry points:

- `BookmarkButton` component — rendered in the article action bar on every article page
- `/account/bookmarks` — the user's saved articles list

2.3 `ingest_jobs` (Migration, inline)

Tracks background index rebuild jobs triggered by `POST /api/ingest`. Written by the Python backend using the service role key.

```
CREATE TABLE ingest_jobs (
    id            UUID PRIMARY KEY,
    status        TEXT NOT NULL,         -- 'queued' | 'running' | 'completed'
| 'failed'
    error_message TEXT,
    started_at    TIMESTAMPTZ,
    completed_at  TIMESTAMPTZ,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

No RLS is applied — this table is written and read exclusively by the backend service role.

2.4 `role_change_log` (Migration 018)

Records every change to a user's role for billing dispute audits and security reviews. A trigger on `user_roles` auto-populates it on every role change.

```
CREATE TABLE role_change_log (
    id          BIGSERIAL PRIMARY KEY,
    user_id     UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
    old_role    TEXT,
    new_role    TEXT NOT NULL,
    changed_by  UUID REFERENCES auth.users(id) ON DELETE SET NULL,
    changed_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    reason      TEXT  -- e.g. 'stripe_webhook', 'admin_override',
'signup_default'
);
```

RLS: Users can read their own history. Service role sees all.

Migration file: `supabase/migrations/018_role_change_log.sql`

---

3. New and Changed API Endpoints

These are **additions and changes** to the endpoint list in `DEVELOPER_GUIDE.md`.

3.1 `POST /api/webhooks/sanity` (Frontend — Next.js)

Receives Sanity content-change webhooks and triggers a RAG re-index.

- **Auth:** HMAC-SHA256 signature in `sanity-webhook-signature` header, verified against `SANITY_WEBHOOK_SECRET`
- **On success:** Calls `POST <PYTHON_BACKEND_URL>/api/ingest` with `Authorization: Bearer $INGEST_SECRET`
- **Response:** `200 { received: true }` or `401` if signature invalid

Setup in Sanity:

1. Go to manage.sanity.io → your project → API → Webhooks
2. Add webhook URL: `https://your-domain.vercel.app/api/webhooks/sanity`
3. Set the secret to match `SANITY_WEBHOOK_SECRET` in your Vercel env
4. Trigger on: Document create / update / delete

3.2 `POST /api/ingest` (Backend — Python/FastAPI) — Updated

This endpoint now returns `202 Accepted` immediately and runs the index rebuild asynchronously in the background. It no longer blocks.

- **Auth:** `Authorization: Bearer $INGEST_SECRET` (open in dev if unset)
- **Conflict handling:** Returns `{ status: "already_running" }` if a job is in progress — safe to retry
- **Response:** `{ status: "accepted", job_id: "<uuid>", message: "..." }`

3.3 `GET /api/ingest/status` (Backend — Python/FastAPI) — New

Poll the status of the most recent index rebuild job.

- **Auth:** Same `Authorization: Bearer $INGEST_SECRET` header
- **Response (idle):** `{ status: "idle", message: "No ingest job has run since last restart." }`
- **Response (active):** `{ job_id, status, queued_at, started_at?, completed_at?, error? }`
- **Possible status values:** `queued` → `running` → `completed` | `failed`

Typical polling pattern:

```
# Trigger rebuild
curl -X POST https://your-backend.railway.app/api/ingest \
  -H "Authorization: Bearer $INGEST_SECRET"

# Poll until completed
curl https://your-backend.railway.app/api/ingest/status \
  -H "Authorization: Bearer $INGEST_SECRET"
```

---

4. AI Pipeline Changes

4.1 Tier-Aware Engine Routing

The `/api/chat` endpoint now routes to two different engines based on user tier. This is **not yet documented** in `DEVELOPER_GUIDE.md`.

Tier	Engine	Behavior
`subscriber`, `student`	`ContextChatEngine`	RAG-only. Retrieves context, streams answer. Faster and cheaper.
`professional`, `advisory`, `admin`	`ReActAgent`	Full agentic loop. LLM can call external tools before answering. Up to 5 reasoning iterations.

Tools available to the agentic pipeline are registered in `backend/services/tools.py` as `ALL_TOOLS`.

4.2 System Prompt Injection Guard

A prompt injection detector runs on every incoming message and `systemPrompt` field before the LLM sees them. Phrases like `"ignore previous instructions"`, `"repeat your system prompt"`, and 7 others are blocked. Blocked messages receive a canned deflection response — no error is surfaced to the user.

4.3 RAG Query Logging

Every chat request (except `user_id == "dev"`) is logged to `rag_query_log` after the response completes. Logged fields include the query, tier, model used, retrieved document IDs and scores, first 500 chars of the response, and latency. This is the primary input for Ragas evaluation (see §6).

---

5. Error Monitoring (Sentry)

Sentry is now integrated in both frontend and backend.

Frontend: Configured via `sentry.client.config.ts`, `sentry.server.config.ts`, and `sentry.edge.config.ts`. Source maps are uploaded at build time using `SENTRY_AUTH_TOKEN`. Errors are captured automatically by the Next.js SDK.

Backend: `sentry-sdk[fastapi]` is in `requirements.txt`. Initialized in `main.py` using the `SENTRY_DSN` environment variable (set in Railway dashboard). FastAPI and Starlette integrations are registered automatically.

Triage workflow:

1. Go to sentry.io → your project → Issues
2. Filter by `environment:production`
3. Click any issue to see the full stack trace, user context, and request payload
4. The `user_id` is attached to each event via the auth middleware

---

6. RAG Evaluation with Ragas

A golden test set and evaluation runner now exist at `backend/eval/evaluate_rag.py`.

Running an evaluation

```
# Install eval-only dependencies (one-time)
pip install -r backend/requirements-eval.txt

# Run from the backend directory
cd backend
python -m eval.evaluate_rag
```

The script:

1. Loads the golden Q&A dataset (defined inline in the file)
2. Runs each question through the live RAG pipeline
3. Scores four metrics via Ragas: **faithfulness**, **answer_relevancy**, **context_precision**, **context_recall**
4. Prints a summary table and saves results to `eval/results_<timestamp>.csv`

Interpreting results

Metric	Target	Meaning if low
`faithfulness`	> 0.85	LLM is hallucinating — not sticking to retrieved context
`answer_relevancy`	> 0.80	Answers are off-topic or vague
`context_precision`	> 0.75	Retrieved chunks are not relevant to the question
`context_recall`	> 0.70	Retrieved chunks are missing information needed to answer

Expanding the test set

The golden dataset is defined in `GOLDEN_DATASET` inside `evaluate_rag.py`. It currently has ~12 Q&A pairs. **Target: 50+ pairs** covering all five pillars (Policy, Economics, Technology, Clinical, Equity) before treating eval results as statistically meaningful.

Add entries in this format:

```
{
    "question": "...",
    "ground_truth": "...",   # expected factual answer
    "pillar": "Policy",      # for filtering/segmentation
}
```

When to run

- Before and after every content ingest to detect regressions
- After changing chunking strategy, retrieval top-k, or re-ranker settings
- Monthly as a baseline health check

---

7. Remaining Manual Configuration (Cannot Be Done in Code)

The following items require action in third-party dashboards and cannot be automated.

7.1 Loops Email Template IDs

`LOOPS_TEMPLATE_*` variables are intentionally blank in `.env.production.example`. Email flows (welcome, digest, payment failed, trial ending) will silently no-op until these are set.

Steps:

1. Log into app.loops.so → Templates
2. Create (or duplicate) a transactional template for each flow
3. Copy each template's ID into your Vercel project environment variables:
    - `LOOPS_TEMPLATE_WELCOME`
    - `LOOPS_TEMPLATE_DIGEST`
    - `LOOPS_TEMPLATE_PAYMENT_FAILED`
    - `LOOPS_TEMPLATE_TRIAL_ENDING`
    - `LOOPS_TEMPLATE_SURVEY_RESULTS`

7.2 `PYTHON_BACKEND_URL` in Vercel Dashboard

Set `PYTHON_BACKEND_URL=https://your-actual-backend.railway.app` in the Vercel project dashboard (Settings → Environment Variables) before the first production deployment. This is the only remaining placeholder that affects live traffic.

---

8. Act 167 Policy Simulation Engine

Added: March 2026 | **Route:** `/vermont-act-167/simulator`

A browser-based policy analysis tool for modeling the implementation scenarios described in the Oliver Wyman "Act 167 Community Engagement: Recommendations" report (August 2024). Full user and technical documentation is in **`ACT167_SIMULATOR_GUIDE.md`** at the project root. This section records only the developer-relevant changes not covered by that guide.

8.1 New Files

File	Purpose
`frontend/app/vermont-act-167/simulator/page.tsx`	Main simulator page — 8-tab interface, all UI components, state management
`frontend/app/vermont-act-167/simulator/data.ts`	All simulation data: 14 hospitals, 18+ recommendations, 14 counties, state benchmarks, pillar scoring
`frontend/app/vermont-act-167/simulator/LeafletMap.tsx`	Leaflet + OpenStreetMap map component — loaded dynamically (`ssr: false`)
`ACT167_SIMULATOR_GUIDE.md`	2,400-line user guide + technical reference for the simulator

The main page at `frontend/app/vermont-act-167/page.tsx` was also updated to add navigation links to the simulator (two `Link` elements pointing to `/vermont-act-167/simulator`).

8.2 New npm Dependencies

Three packages were added to `frontend/package.json`:

Package	Version	Purpose
`leaflet`	`^1.9.4`	Core Leaflet map library
`react-leaflet`	`^5.0.0`	React bindings for Leaflet
`@types/leaflet`	(devDep)	TypeScript types for Leaflet

`d3-geo` (`^3.1.1`) and `@types/d3-geo` were already present and are used by other components; the simulator originally used them for a custom SVG map but that was replaced by Leaflet. The `d3-geo` import was removed from `simulator/page.tsx` — no package removal needed.

8.3 Content Security Policy Change (Critical)

File: `frontend/next.config.ts`

The `img-src` directive in the CSP header was updated to allow OpenStreetMap tile images. Without this change the Leaflet map renders as a gray rectangle — tiles load but are blocked before display.

Before:

```
img-src 'self' blob: data: https://cdn.sanity.io https://img.youtube.com
```

After:

```
img-src 'self' blob: data: https://cdn.sanity.io https://img.youtube.com
https://*.tile.openstreetmap.org
```

The wildcard covers all three OSM tile subdomains (`a.`, `b.`, `c.tile.openstreetmap.org`). No other CSP directives were changed — OSM tiles are images only, not scripts or fetch targets.

> **Note for deployment:** This CSP change takes effect on next server start. In production (Vercel), it is applied automatically on the next deploy. If you switch tile providers in the future (e.g., Mapbox, Stamen), add their image domain here.

8.4 SSR Handling for Leaflet

Leaflet accesses `window` and `document` at import time and cannot run server-side. The `LeafletMap` component is therefore loaded with Next.js dynamic import:

```
// In simulator/page.tsx
const LeafletMap = dynamic(() => import("./LeafletMap"), { ssr: false });
```

`LeafletMap.tsx` itself is marked `"use client"` but that alone is insufficient in App Router — the `dynamic(..., { ssr: false })` wrapper is required to prevent Node.js from evaluating the Leaflet import during SSR. If you refactor this component, keep the dynamic import; removing it will cause a build-time `ReferenceError: window is not defined`.

8.5 Simulator Architecture Summary

The simulator is entirely client-side and stateless — no backend calls, no database reads, no authentication required. All data is embedded in `data.ts` as TypeScript constants (synthetic + Wyman Report figures). The tab layout and all inter-component state live in the `Act167SimulatorPage` default export in `page.tsx`.

The geographic map tab (`🗺️ Geographic Map`) uses:

- **OpenStreetMap** tiles via Leaflet (free, no API key)

- A hard-coded Vermont state border polygon (`VT_BORDER` in `LeafletMap.tsx`) — 33 [lat, lng] coordinate pairs tracing the actual perimeter
- `CircleMarker` elements for hospitals, sized by bed count and colored by the active analysis layer (urgency / financial health / equity risk / access & travel time)
- Optional `Polyline` overlay for COE network links (UVMMC hub → spoke hospitals)
- Optional `Circle` bubbles for HSA population size

8.6 HubPageTemplate Integration and Tab Wrapping Fix

The simulator's custom header and tab navigation were replaced with the shared `HubPageTemplate` component (used by Academy and other hub pages) to ensure visual consistency across the platform. This involved:

- **Removing** the dark-gradient custom header and the pill-style tab bar from `simulator/page.tsx`
- **Adding** imports for `HubPageTemplate`, `useRouter`, and nine Heroicons (one per tab)
- **Rebuilding** the main component to pass tab content as `React.ReactNode` to `HubPageTemplate`
- **Using** `router.push("?tab=roadmap", { scroll: false })` for the programmatic "View Implementation Roadmap →" tab switch (previously `setActiveTab("roadmap")`)
- **Moving** the Select All / Clear / View Roadmap quick-controls into the `ScenarioBuilder` component

`HubPageTemplate` itself was also patched in the same session (`components/templates/HubPageTemplate.tsx`): the tab `<nav>` changed from `overflow-x-auto hide-scrollbar` to `flex-wrap` so tabs wrap to a second row on narrow screens instead of scrolling horizontally. This fix applies globally to all pages using the template (Academy, simulator, etc.).

8.7 Replacing Synthetic Data with Real Data

The simulator currently uses synthetic projections where actual data is unavailable. See **`ACT167_SIMULATOR_GUIDE.md` §20 (Data Ingestion Guide)** for a full inventory of what real data is needed, where to obtain it (GMCB, AHS, Census), and how to update `data.ts`. No backend or API changes are required — data replacement is a `data.ts` edit only.

---

9. HTR Simulator Hub

Added: March 2026 | **Route:** `/htr-simulator`

A standalone, top-level hub page that elevates the simulation engine as a major platform feature independent of any single state initiative. Vermont Act 167 is now positioned as a *use case* of the generic HTR Simulator framework rather than its only home.

9.1 Architectural Relationship

Layer	Route	Purpose
HTR Simulator Hub	`/htr-simulator`	Generic, educational, framework documentation. Overview, 5-Pillar Framework, Simulation Engine mechanics, Use Cases index, Methodology. No state-specific data.
Act 167 Use Case	`/vermont-act-167/simulator`	Vermont-specific simulation. 14 hospitals, 18+ recommendations, VT data, geographic map. Links back to hub.
CalAIM (planned)	`/california-calaim/simulator`	California Medi-Cal transformation — Coming Soon.
Oregon CCO (planned)	`/oregon-cco/simulator`	Oregon Coordinated Care Organizations — Coming Soon.
CMS Rural Health (planned)	`/dashboard/simulator`	Federal rural health transformation — Coming Soon.

9.2 New Files

File	Purpose
`frontend/app/htr-simulator/page.tsx`	Full 5-tab hub page — all content self-contained, no external data dependencies

9.3 HomeSidebar Change

`frontend/components/HomeSidebar.tsx` was updated to add an **HTR Simulator** entry at the top of the amber **Tools & Resources** card. It uses `CpuChipIcon` from `@heroicons/react/24/outline` (newly imported). The entry links to `/htr-simulator` and includes an active-state highlight consistent with other sidebar items.

9.4 HubPageTemplate: `rowBreakAfter` Prop

`frontend/components/templates/HubPageTemplate.tsx` received a new optional prop `rowBreakAfter?: number`. When set, a `w-full` flex break is inserted after the nth tab, forcing a controlled two-row layout. Used by the Act 167 simulator (`rowBreakAfter={4}`) to split its 9 tabs into a 4+5 arrangement. Other pages using `HubPageTemplate` are unaffected (prop defaults to `undefined`).

9.5 HTR Simulator Tab Structure

Tab	Content Summary
Overview	What HTR Simulator is, 6 capability cards, who it's for (3 audiences), CTA to Vermont Act 167
5-Pillar Framework	Full explanation of all 5 pillars with scored dimensions and score interpretation guide (0–100 scale)
Simulation Engine	3-stage process (recommendation scoring → scenario aggregation → impact projection), data model, limitations
Use Cases	Cards for all 4 configured use cases (1 live, 3 coming soon) plus "propose a use case" CTA
Methodology	Scoring dimensions, data sources (CMS, AHA, Census, HIFLD), key assumptions, version note

---

10. UI Standardisation — Tabs Everywhere

Updated: March 2026

All major hub and feature pages now use the same `HubPageTemplate` browser-tab style for navigation. Cards-to-tabs conversions and new components are recorded here.

10.1 Tab Style Standard

The platform tab standard is `HubPageTemplate` (`frontend/components/templates/HubPageTemplate.tsx`). All pages that present multiple views must use this component or match its exact button classes:

State	Classes
Active tab	`bg-slate-100 border-black text-slate-900 z-10 -mb-px rounded-t-xl border-t border-l border-r`
Inactive tab	`bg-white border-slate-200 text-slate-500 hover:bg-slate-50 hover:text-slate-700 mt-1.5 shadow-sm rounded-t-xl border-t border-l border-r`
Nav container	`flex flex-wrap justify-center items-end border border-slate-200 rounded-t-xl px-2 bg-slate-50/80 backdrop-blur-sm pt-2 gap-y-1`

State	Classes
Sticky wrapper	`sticky z-30 mb-8` with `top: var(--sidebar-top, 8.5rem)`

10.2 Research Lab — Unified Two-Level Tab Interface

The Research Lab (`/research-lab`) was rebuilt from a card-grid landing page + 6 separate sub-section pages into a single unified page with a two-level sticky nav.

Architecture change:

Before	After
`/research-lab` — 6 clickable cards → navigate to sub-page	`/research-lab` — full two-level tab experience, no sub-page navigation needed
`/research-lab/[section]` — primary entry point, stacked card tools	`/research-lab/[section]` — still valid for direct linking, uses `LabPageShell`

Two-level nav design:

- **Row 1 — Section tabs**: 6 sections using standard `HubPageTemplate` browser-tab style. Active state synced to URL `?tab=` param for bookmarkability.
- **Row 2 — Tool pills**: Tool sub-selection using pill/chip style (`rounded-full`, `text-xs`, filled dark on active). Visually subordinate to section tabs — smaller, different shape, different active treatment — to prevent confusion when both rows show an active state simultaneously.
- Both rows live inside the **same** `<nav>` container with a `<div className="w-full" />` forced break between them, so the `gap-y-1` spacing matches advisory-hub's naturally-wrapped tabs exactly.

New / changed files:

File	Change
`frontend/app/research-lab/ResearchLabHub.tsx`	New — purpose-built component with two-level sticky nav and all 19 dynamic tool imports
`frontend/app/research-lab/page.tsx`	Updated — server component wrapper (auth/upgrade prompts) that renders `ResearchLabHub`
`frontend/components/templates/SubTabView.tsx`	New — reusable secondary tab component matching `HubPageTemplate` button style; available for future nested-tab use cases
`frontend/app/research-lab/ResearchLabTabs.tsx`	**Deleted** — superseded by `ResearchLabHub`

10.3 Advisory Hub — Cards to Tabs

`/advisory-hub` was converted from a 10-card grid to a `HubPageTemplate` page with 9 practice-area tabs.

Each tab panel shows the practice area's description, pillar tags, a link to the full service sub-page, and a "Book a Discovery Call" CTA. The "Start an Engagement" card was removed as a separate card; its CTA is now embedded in every tab panel.

New / changed files:

File	Change
`frontend/app/advisory-hub/AdvisoryHubClient.tsx`	New — `'use client'` component with `HubPageTemplate`, 9 tabs, `ServicePanel` renderer

File	Change
`frontend/app/advisory-hub/page.tsx`	Updated — thin server wrapper (metadata export) that renders `AdvisoryHubClient`

10.4 Dead Code Removed

File	Reason
`frontend/lib/data/rht-awards.ts`	185 lines, zero imports anywhere in the codebase
`frontend/lib/data/states.ts`	Superseded by `frontend/lib/db/states.ts` (Supabase fetcher); confirmed zero active imports

---

End of addendum. Merge into respective guides during next documentation pass.